



Anatomía del agente

Guía 5 · Taller 02 de caso aplicado de IA en industria

De un bucle plano a un grafo con siete nodos, y por qué esa diferencia importa.

Primero, el bucle plano

Veinte líneas. Está en `agente/plano.py` y se corre con `make plano`.

```
llm = obtener_llm().bind_tools(HERRAMIENTAS)
mensajes = [SystemMessage(content=CONTEXTO), HumanMessage(content=encargo)]

for _ in range(MAX_VUELTAS):
    respuesta = llm.invoke(mensajes)
    mensajes.append(respuesta)
    if not respuesta.tool_calls:
        break
    for llamada in respuesta.tool_calls:
        resultado = POR_NOMBRE[llamada["name"]].invoke(llamada["args"])
        mensajes.append(ToolMessage(str(resultado), tool_call_id=llamada["id"]))
```

Esto es ReAct entero: pensar, actuar, observar, repetir. Y **funciona**: llama las herramientas correctas sin que nadie le diga cuáles, encadena, encuentra cosas de verdad y escribe un diagnóstico que cualquiera firmaría.

El problema aparece al correrlo tres veces sobre el mismo mundo.

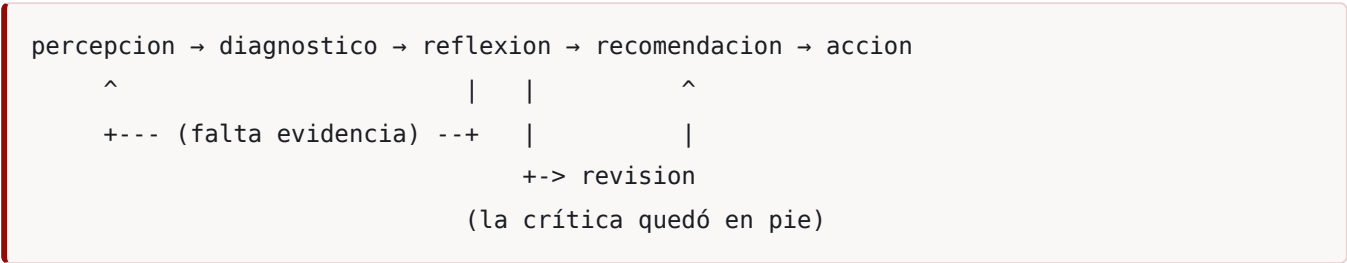
Las cuatro patologías

QUÉ HACE	POR QUÉ PASA
Concluye con lo primero que encuentra	Nada le dice cuándo tiene suficiente
Ordena por la señal más ruidosa	El MAPE tiene los números más grandes
Dramatiza el impacto de negocio	Ninguna cifra tiene que defenderse
Mañana redescubre lo mismo	No hay memoria entre ejecuciones

Ninguna es del modelo. **Todas son de la forma del bucle.**

Y ninguna se arregla con un prompt más largo: quince reglas se cumplen a medias y cuáles se olvidan cambia en cada ejecución; no hay forma de comprobar que las cumplió; y ninguna instrucción hace que la ejecución de mañana sepa lo que dijo la de hoy.

El grafo



NODO	SU ÚNICO TRABAJO
percepcion	Decide qué mirar y llama herramientas. No concluye nada
diagnostico	Convierte la evidencia en una hipótesis, con alcance y severidad
reflexion	Intenta refutar su propia hipótesis
revision	Reescribe el diagnóstico haciéndose cargo de la crítica
recomendacion	Emite acciones concretas
accion	Ejecuta las que una política de código deja pasar. Sin LLM

Lo único que hay que señalar del dibujo son **las dos flechas que vuelven hacia atrás**. Todo lo demás es cableado; esas dos son lo que lo convierte en algo que se puede creer.

Por qué reflexión y revisión, y no solo una

Cuestionarse sin poder corregirse no sirve de nada: el diagnóstico saldría contradiciendo a sus propias objeciones. Por eso son dos nodos.

La decisión de a dónde ir después **no la toma el LLM**:

```
def _tras_reflexion(estado) -> Literal["percepcion", "revision", "recomendacion"]:
    if estado["critica"].get("veredicto") != "insuficiente":
        return "recomendacion"
    if estado.get("vueltas", 0) < MAX_VUELTAS:
        return "percepcion"
    return "revision"
```

La toman un `if` y un contador de vueltas sobre un campo que el LLM escribió. Esa distinción es el reto entero.

Una capa no es un prompt

Es **un nodo con un solo trabajo, que corre o no corre y deja rastro**. A un prompt que dice «*no dramáticas*» no se le puede preguntar si lo cumplió. A un nodo de reflexión sí: o produjo objeciones o no las produjo, y están escritas.

El freno

Cuando el agente puede actuar, equivocarse deja de costar una alerta y pasa a costar 24 modelos reentrenados con datos malos. La política vive en `agente/accion.py`, es **código Python**, y no hay forma de convencerla:

- **Regla 1 · la naturaleza del problema.** Si el diagnóstico es una anomalía, el problema está en los datos. Reentrenar ahí enseña ruido.
- **Regla 2 · el radio de daño.** Reentrenar una categoría toca 24 modelos y se revierte. Reentrenar la flota toca los 192 a la vez, y si el diagnóstico estaba mal **no queda ningún modelo sano contra el cual comparar**.

Y viene apagado: sin `EJECUTAR_ACCIONES=1` evalúa qué haría y no hace nada.

El agente propone; estas reglas disponen. Y nunca frenes contra un campo que el propio agente redacta: si el freno depende de algo que el modelo escribe, el modelo puede aflojarlo sin querer.
